STM32

1 Background

Microcontrollers or MCUs for short, are devices which run a specific set of instructions to interact with hardware peripherals to control signals in a circuit. These peripherals include signaling protocols like I^2C, SPI, UART/USART, CAN; simple logic devices like Timers, Comparators, Cryptographic Accelerators; and low-level hardware interfaces like GPIO, ADCs, and DACs. For us on the team, we use MCUs to handle any logic needed to drive electronic components on the car. Some examples include: Driver Dashboard for the IC, DRS control board, BMS for the EV, etc... Basically, anytime we need code to run on the car, we use MCUs.

There are a handful of major MCU companies:

- Microchip (Including Atmel)
- NXP Semiconductors
- Infineon Technologies
- Texas Instruments
- Silicon Labs
- STMicroelectronics

2 STMicroelectronics

As a team, we have standardized the use of STMicroelectronics (ST) MCUs. Their devices are low cost, simple to develop and have good documentation. Additionally, their development boards are beginner friendly with a similar structure to an Arduino. ST's main line of MCUs is their STM32 family of products. Within this family, many different sub-families exist including F, G, U, H, N, W, C, and MP. Each of these are oriented around a specific use case, for example their H line is for high-performance applications, and their U line is for low power applications. For our use cases, we use their F family of MCUs which are oriented to general usage.

Beyond these subfamilies, they have different tiers of each, this defines the amount of memory, types of peripherals and clock speed. See their website for more information about their naming structure and the various features of each MCU.

As I said before, they are generally beginner friendly as long as you are pointed in the correct direction, the rest of this guide is to introduce you to the tools, workflow and some basic functions of STM32s to hopefully allow you to find what you need in the future more effectively, and most importantly, without too much help from AI while still following best practices.

2.1 A Comment About AI

Personally, I have no gripe against AI. I feel like it is a phenomenal tool to be able to quickly find what you need and help you parse through a lot of information. Additionally, it is a great tool to find out what you need to be looking at to help you out (the way I think of it is: if you don't know what to search for, how do you find the information you need). AI excels at getting you started in a direction (not always in the right direction, but a direction).

A warning though: AI is trained mostly on code written for normal computers, thus it knows much more about that than embedded work. Every time that I have used AI for embedded coding, I end up having to correct simple mistakes and small errors that are not the most intuitive thing to notice. Here's a small list of errors AI has made (maybe my prompting sucks, but still some examples): I've had it

- say that a peripheral exists when it doesn't
- make up functions that don't exist
- say a function should return a value, when it doesn't
- sign off on code which was missing entire functions needed to work

Additionally, because hardware constraints can be unique, the AI will not know everything about your setup and thus, completely fudge things up. Blinding trusting AI written code is a quick way to end up with a rat's nest of code you have no idea how to work on and doesn't function. Unlike regular coding, which it can do well, it generally sucks for embedded. So, my general advice: use AI as a tool to **help** you get to the end result, but not as a tool to **give** you the end result. Critique everything produced and assume it's wrong until you prove otherwise. But back to STM32s.

2.2 Software

ST provides a suite of software to program the STM32 MCU, they are as follows:

- **STM32CubeMX**: Performs project creation and provides a GUI to configure STM32 MCU peripherals and settings.
  https://www.st.com/content/st_com/en/products/development-tools/software-development-tools/stm32-software-development-tools/stm32-configurators-and-code-generators/stm32cubemx.html

- **STM32CubeIDE**: An all-in-one Integrated Development Environment configured specifically for STM32 MCUs. This IDE has development, debug and programming tools.
  https://www.st.com/en/development-tools/stm32cubeide.html

- **STM32CubeProgrammer**: A standalone tool which allows for management of onboard memory including programming, erasing and downloading of memory contents.
  https://www.st.com/content/st_com/en/products/development-tools/software-development-tools/stm32-software-development-tools/stm32-programmers/stm32cubeprog.html

For development, CubeProgrammer is not needed as CubeIDE performs the needed programming operations. Thus, CubeMX and CubeIDE are the only pieces of software which need to be downloaded and installed.

Note: CubeProgrammer provides a more intuitive interface for managing the memory of an MCU but is mostly redundant with CubeIDE for development purposes. Feel free to look into CubeProgrammer if interested, but it's not necessary.

2.2.1 STM32CubeMX Download & Installation

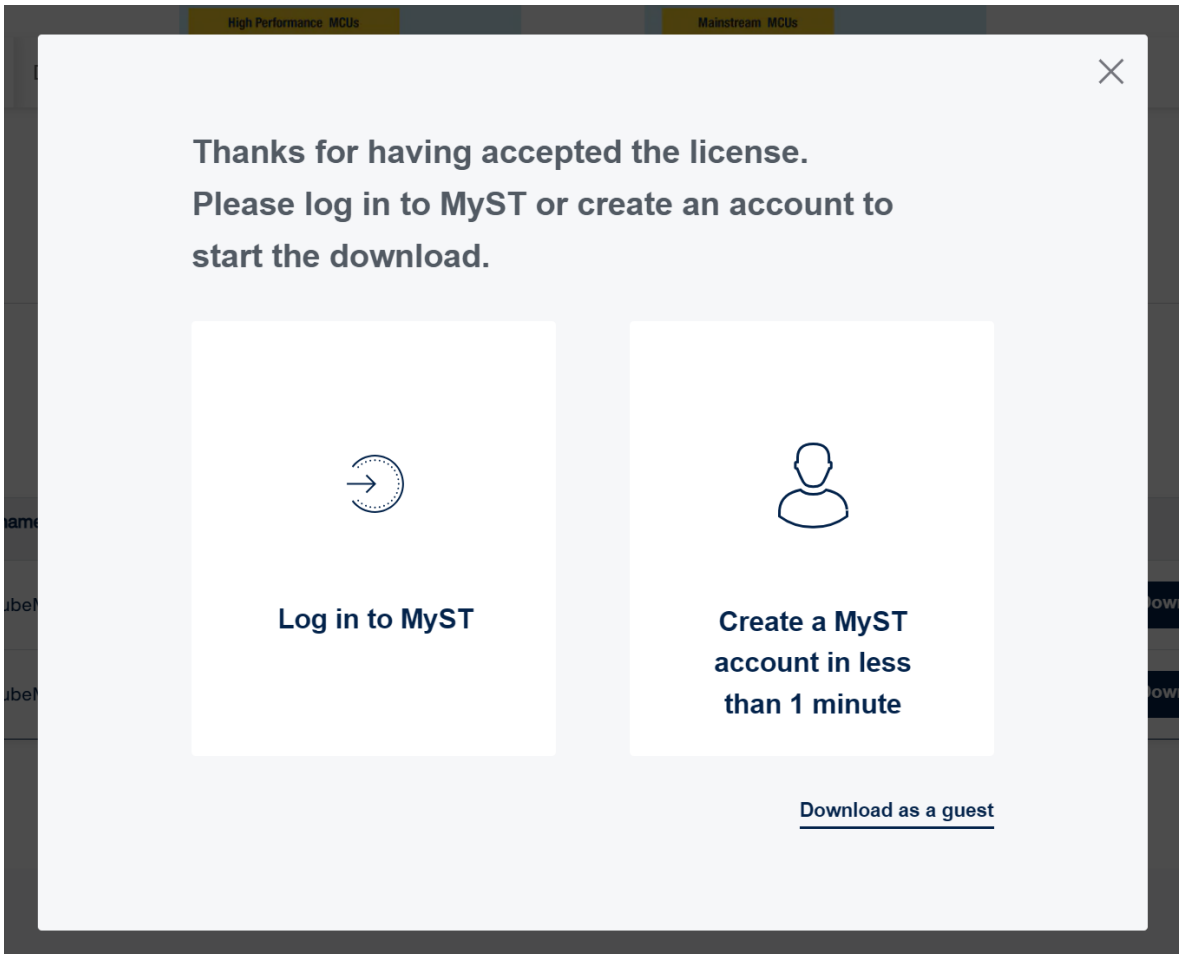
- Navigate to the download page: https://www.st.com/content/st_com/en/products/development-tools/software-development-tools/stm32-software-development-tools/stm32-configurators-and-code-generators/stm32cubemx.html

- Scroll down to the '**Get Software**' section and next to STM32CubeMX, select the correct OS and latest version. Click '**Download**'. Do not download STM32CubeMX2.

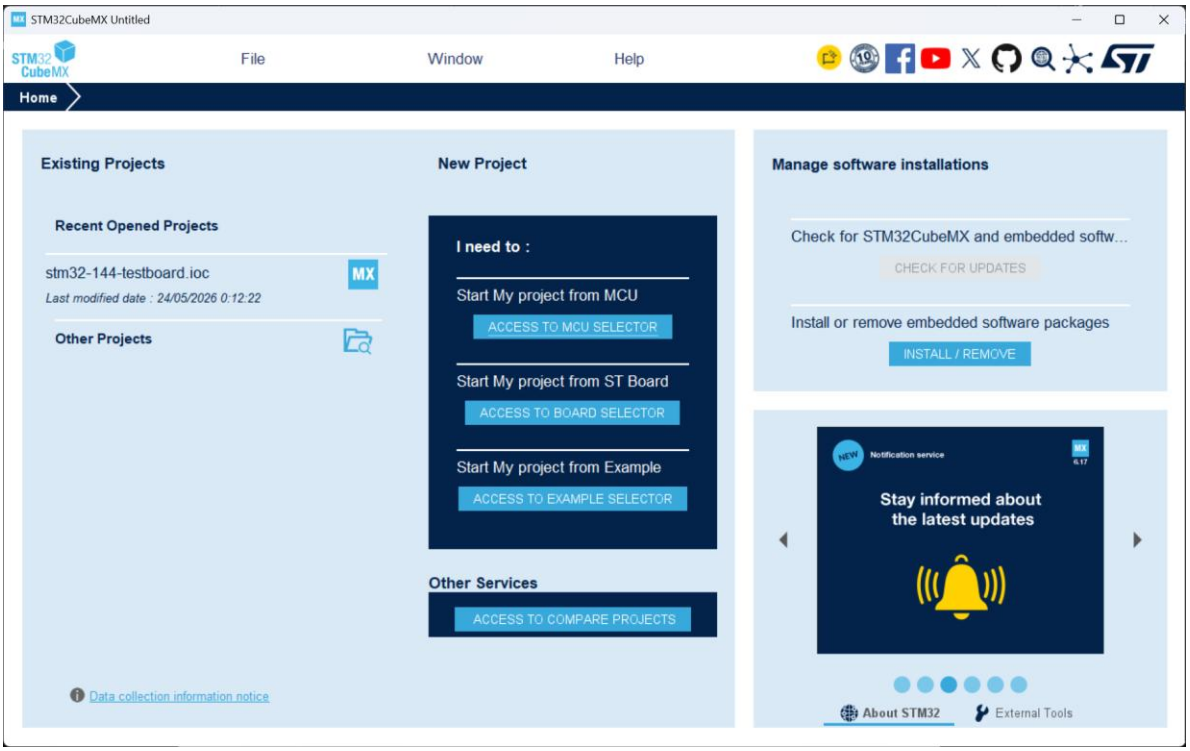


-

- Read and accept the Terms and Conditions. After accepting, you must either create/login to a ST.com account, or fill out the download form under '**Download as a guest**'.
  ST.com accounts are free and they are good about not spamming your inbox, personally, I would suggest having one to not have to fill out the download form each time, but that is personal preference.



- Depending on which way you decide to provide your information, they will either email you a link or start downloading a zip file. Either way, once you have the zip file, open the included executable.
- Select whichever install mode you want. Push yes for any UAC prompts and follow through the installer. Don't worry about creating an automatic installation script if it asks. If you search for '**STM32CubeMX**' and run the program, you should see the following screen, this concludes the installation.

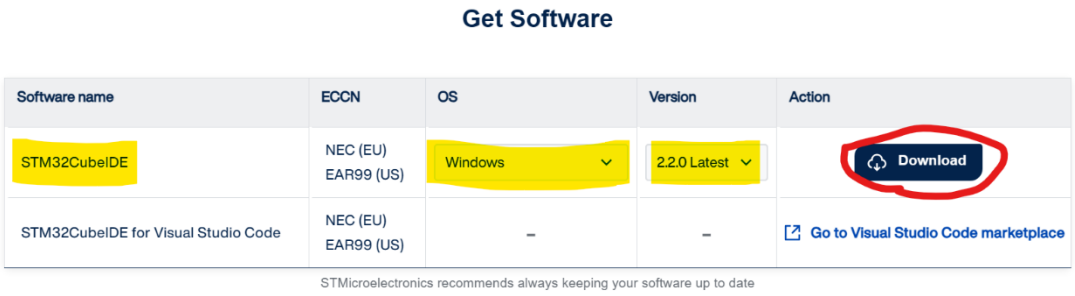


-

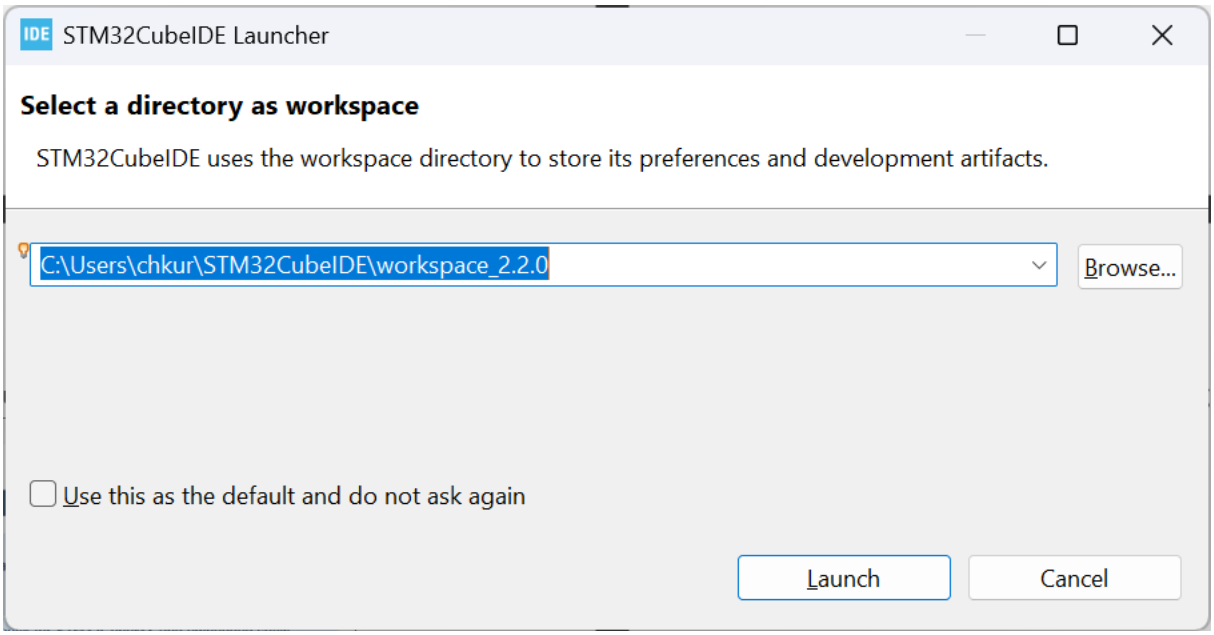
- Close the program and proceed to install STM32CubeIDE.

2.2.2 STM32CubeIDE Download & Installation

- Navigate to the download page: https://www.st.com/en/development-tools/stm32cubeide.html
- Scroll down to the '**Get Software**' section and next to STM32CubeIDE, select the correct OS and latest version. Click '**Download**'.
  Note: I would suggest against using the VSCode extension for now, the full IDE has more stuff and is better supported at the moment. Also, most people tend to use the full IDE, but feel free once you get the hang of it.



- Read and accept the Terms and Conditions. After accepting, you must either create/login to a ST.com account, or fill out the download form under '**Download as a guest**'.
- Depending on which way you decide to provide your information, they will either email you a link or start downloading a zip file. Either way, once you have the zip file, open the included executable.
- Push yes for any UAC prompts and follow through the installer. Ensure to also install the drivers when prompted. Don't worry about creating an automatic installation script if it asks. If you search for '**STM32CubeIDE'** and run the program, you should see the following screen, this concludes the installation.



2.3 Documentation

ST provides a ton of documentation for each of their products and software, for this guide we will focus on the Nucleo-F303RE board, but similar documentation exists for each product. The most useful documents are below, but many more exist on the ST site including specific application notes and other things to help development.

2.3.1 STM32F303RE MCU IC

These documents are specific to this chip (or family), they are written to be generic of the board the chip is on. The main product page is here: https://www.st.com/en/microcontrollers-microprocessors/stm32f303re.html

- **STM32F303RE Datasheet**: This document primarily outlines the features of the MCU, along with the physical and electrical characteristics, in addition to the pinout.
  https://www.st.com/resource/en/datasheet/stm32f303re.pdf

- **RM0316 Reference Manual**: This document outlines how to use the MCU and all the peripherals it has. This is more function rather than actual code and provides in-depth operation descriptions. This document does have register mappings too for working with individual registers if desired.
  https://www.st.com/resource/en/reference_manual/rm0316-stm32f303xbcde-stm32f303x68-stm32f328x8-stm32f358xc-stm32f398xe-advanced-armbased-mcus-stmicroelectronics.pdf

- **UM1786 User manual**: This document describes the functions used to program the MCU in C/C++. This includes both HAL and LL functions including function prototypes, parameters, return values and additional comments.
  https://www.st.com/resource/en/user_manual/um1786-description-of-stm32f3-hal-and-lowlayer-drivers-stmicroelectronics.pdf

- *Honorable Mention:* **PM0214 Programming manual**: This document outlines the actual assembly instructions used by the Arm M4 cores on the chip. If you happen to need to look at compiled code and want to know specific instructions or syntax, this is your doc, otherwise, this isn't totally useful as it is so low level.

2.3.2 Nucleo-F303RE Dev Board

These documents talk about this dev board (or family of dev boards) and less about the actual chip. The main product page is here: https://www.st.com/en/evaluation-tools/nucleo-f303re.html

- **Nucleo-F303RE Data brief**: This document has some basic feature information, otherwise not a lot. https://www.st.com/resource/en/data_brief/nucleo-f303re.pdf

- **UM1724 User manual**: This document has all of the board specific information including hardware, solder bridges, jumpers, LED and Button connections along with header pinouts. https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf

2.3.3 STM32 Software

These documents are about the software packages we installed above. The links above to download are the same ones as the product page.

- **UM1718 User manual (STM32CubeMX)**: This is the main user manual for all of the features of CubeMX. This includes all of the initialization and option configurations. Any information about CubeMX is here. https://www.st.com/resource/en/user_manual/um1718-stm32cubemx-for-stm32-configuration-and-initialization-c-code-generation-stmicroelectronics.pdf

- **UM2609 User manual (STM32CubeIDE)**: This is the user manual for everything in CubeIDE including debugging, setting up the IDE and whatnot. Anything that has to do with the IDE is in this. https://www.st.com/resource/en/user_manual/um2609-stm32cubeide-user-guide-stmicroelectronics.pdf

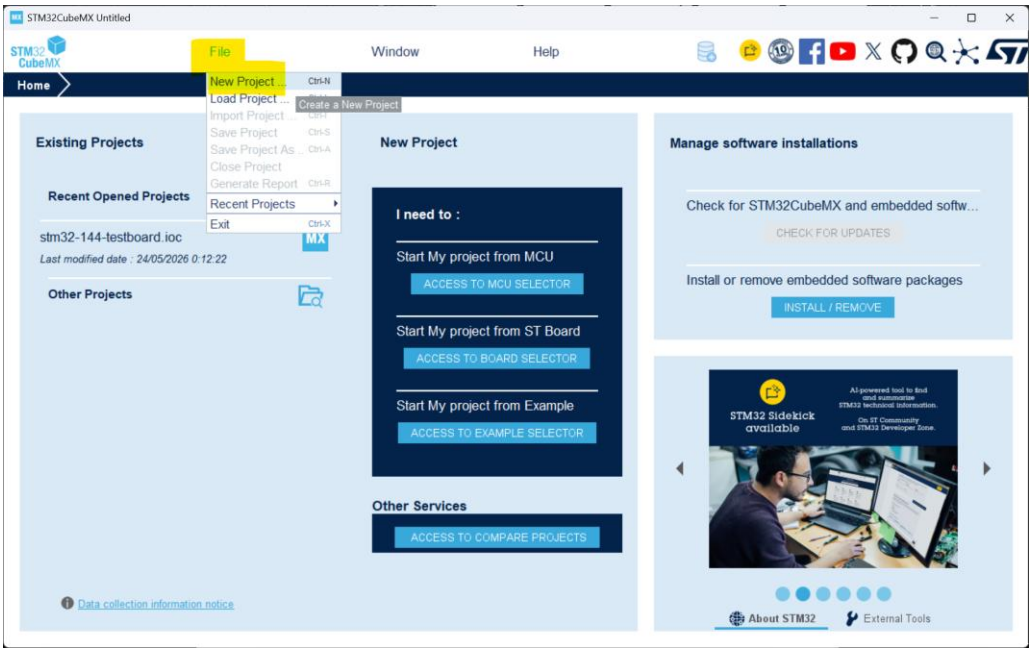
3 Creating Your First STM32 Project: Blinking light

Let's begin with a simple blinking light project and expand on it with some extra features later. For now, we will toggle an on-board LED every 0.5s.

Note: All the information for the project is included here, but if looking to dive deeper, document notations are provided to find where the information came from. The format is ([Document] p[Page_Number]) and I highly recommend looking through them to understand where to find the associated information for future projects.

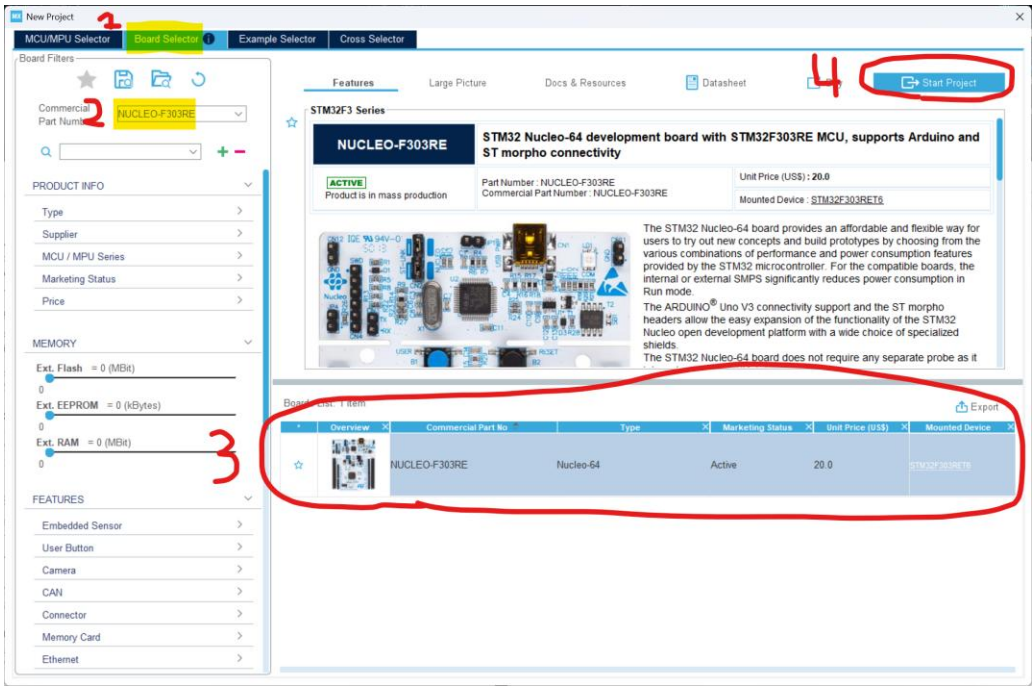
3.1 Creating the Project

Start by opening CubeMX and selecting '**File**->**New Project**'



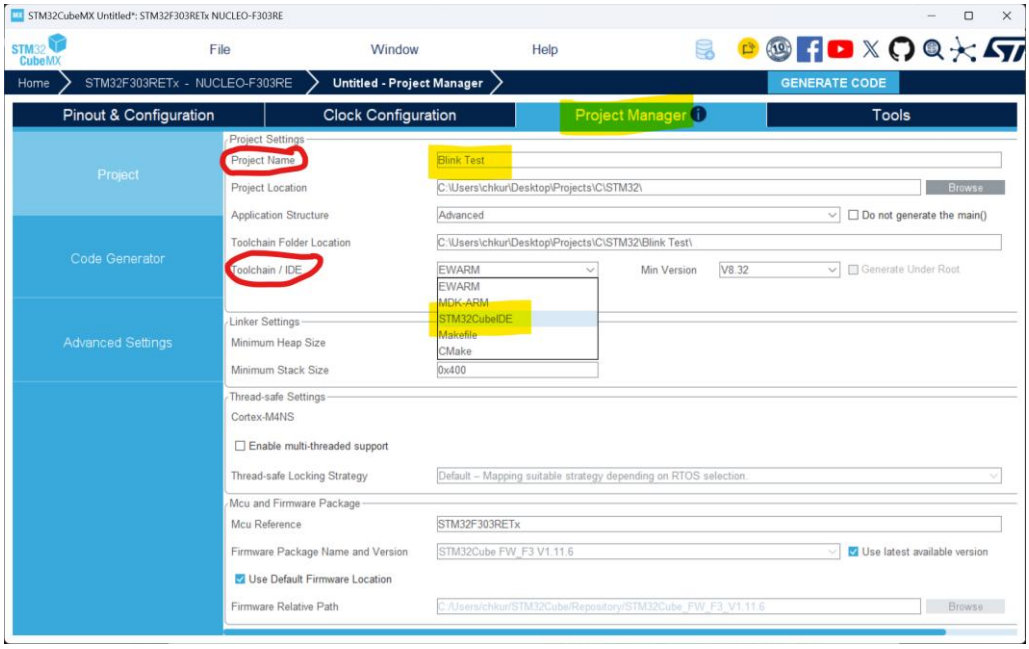
Note: If this is your first time using CubeMX in a while, each step in creating the project might take longer than normal as it needs to download chip and board information.

Then, choose '**Board Selector**', input your Nucleo board into the '**Part Number**' search bar, select the board in the list below and click '**Start Project**'. *Note that if you are working with a custom board, you should go through the '**MCU/MPU Selector**' as selecting using the board will allow CubeMX to know where onboard components like LEDs and Buttons are hooked up to. On a custom board, this may be different and thus CubeMX will not know.*



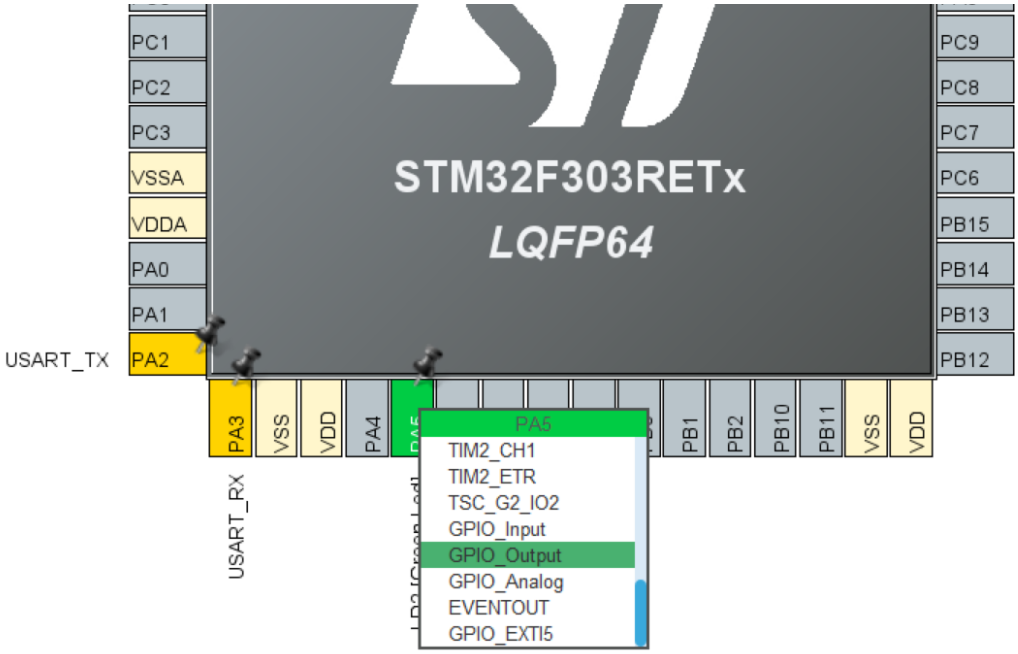
After clicking Start Project, a dialog comes up which asks you if you want to initialize all peripherals to their default modes. Because we selected a Nucleo, this is asking if you want to pre-configure any onboard components. For this tutorial, we will select '**Yes**' as this will pre-configure onboard peripherals like the clock and the GPIO pins.

The main screen pops up; this is where we can do all of the configuration with the project. Start by clicking '**Project Manager**', and input in a project name and change the '**Toolchain / IDE**' to '**STM32CubeIDE**'. Save the project.



3.2 CubeMX Pin Configuration

Next, go back to '**Pinout & Configuration**'. This is you can modify the settings for each pin, and the associated peripheral settings. This is also where you can label pins. If we left click PA5 (where the Green LED is), we can see all the different functions this pin can perform. If you go towards the bottom of the list, you can see that '**GPIO_Output**' is selected, meaning that this pin will act as a controllable, basic output of the MCU.



*Note: Each pin can only have one assignment, meaning that by having this configured to be '**GPIO_Output**' we cannot have it act as '**GPIO_Input**'. But many of these options exist on multiple pins, so if you have two peripherals*

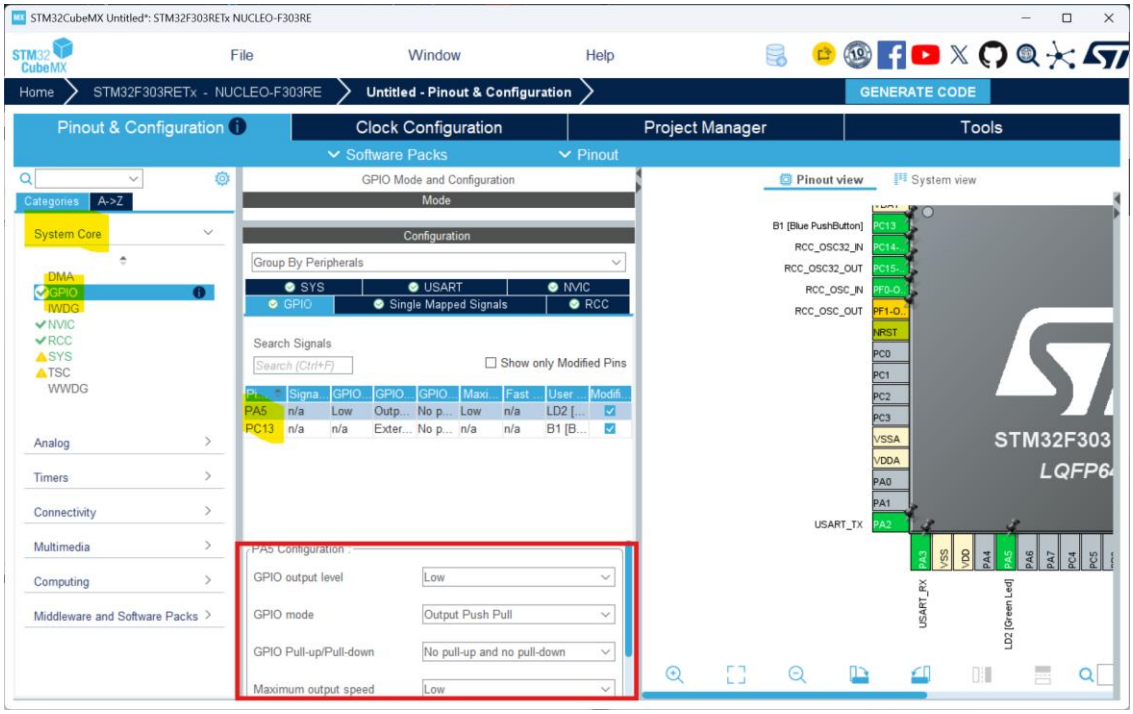
you'd like to use that occupy the same pin, see if there is another pin that can be assigned to that option as well. If we look at the following table from the datasheet, we can see all the functions this pin can do; using the datasheet can be much quicker to CTRL + F to find another pin rather than clicking through each in CubeMX.

Table 13. STM32F303xD/E pin definitions (continued)

Pin number					Pin name (function after reset)	Pin type	I/O structure	Notes	Alternate functions	Additional functions
LQFP64	LQFP100	UFBGA100	WLCSP100	LQFP144						
21	30	K4	H7	41	PA5	I/O	TTa	(5)	TIM2_CH1/TIM2_ETR, TSC_G2_IO2, SPI1_SCK, EVENTOUT	ADC2_IN2(3), DAC1_OUT2, COMP1_INM, COMP2_INM, COMP3_INM, COMP4_INM, COMP5_INM, COMP6_INM, COMP7_INM, OPAMP1_VINP, OPAMP2_VINM, OPAMP3_VINP

(STM32F303RE Datasheet, p44)

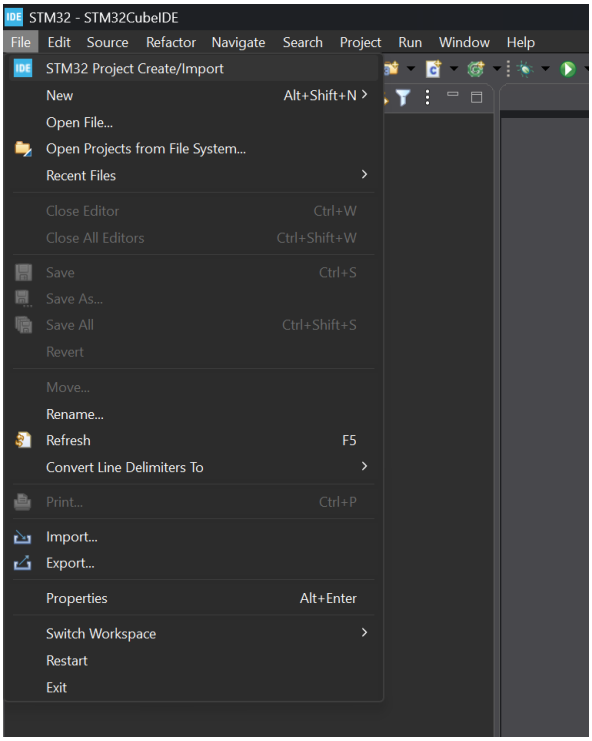
Click on the left side '**System Core**' and select '**GPIO**'. Then, under '**Configuration'**, click the two preconfigured GPIO for the button and LED, you can see the various settings that a GPIO can be configured to. Note that the options will change based on whether the pin is in input or output mode. Leave all the settings as is on this screen.

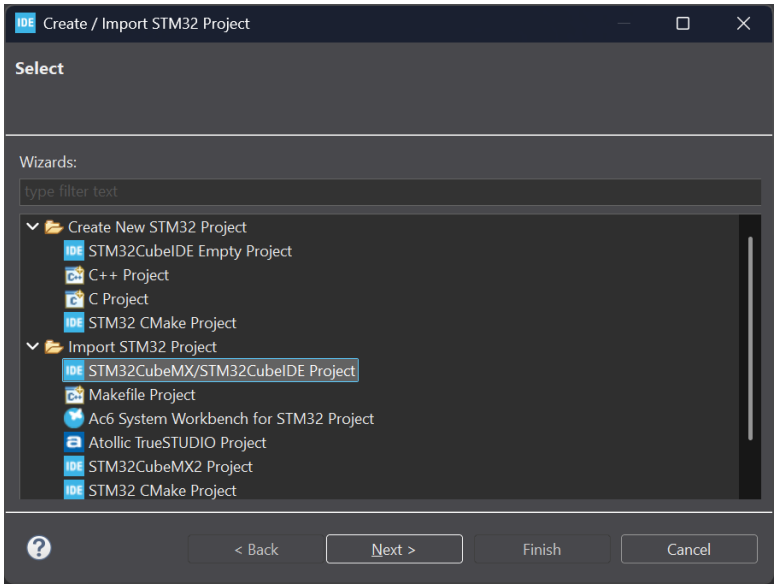


Once complete, click '**Generate Code**'. Ensure you click allow on any UAC prompts that may appear.

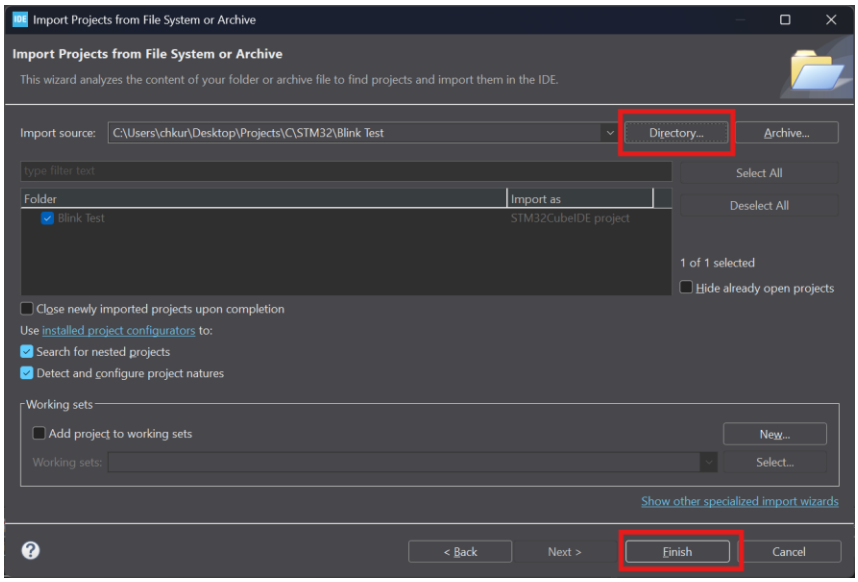
3.3 CubeIDE Project Import from CubeMX

Open CubeIDE and choose the workspace to be the parent folder of the project folder. Click '**File**' and click '**STM32 Project Import/Create**'. Then select under '**Import STM32 Project**', '**STM32CubeMX/STM32CubeIDE Project**' and click '**Next**'

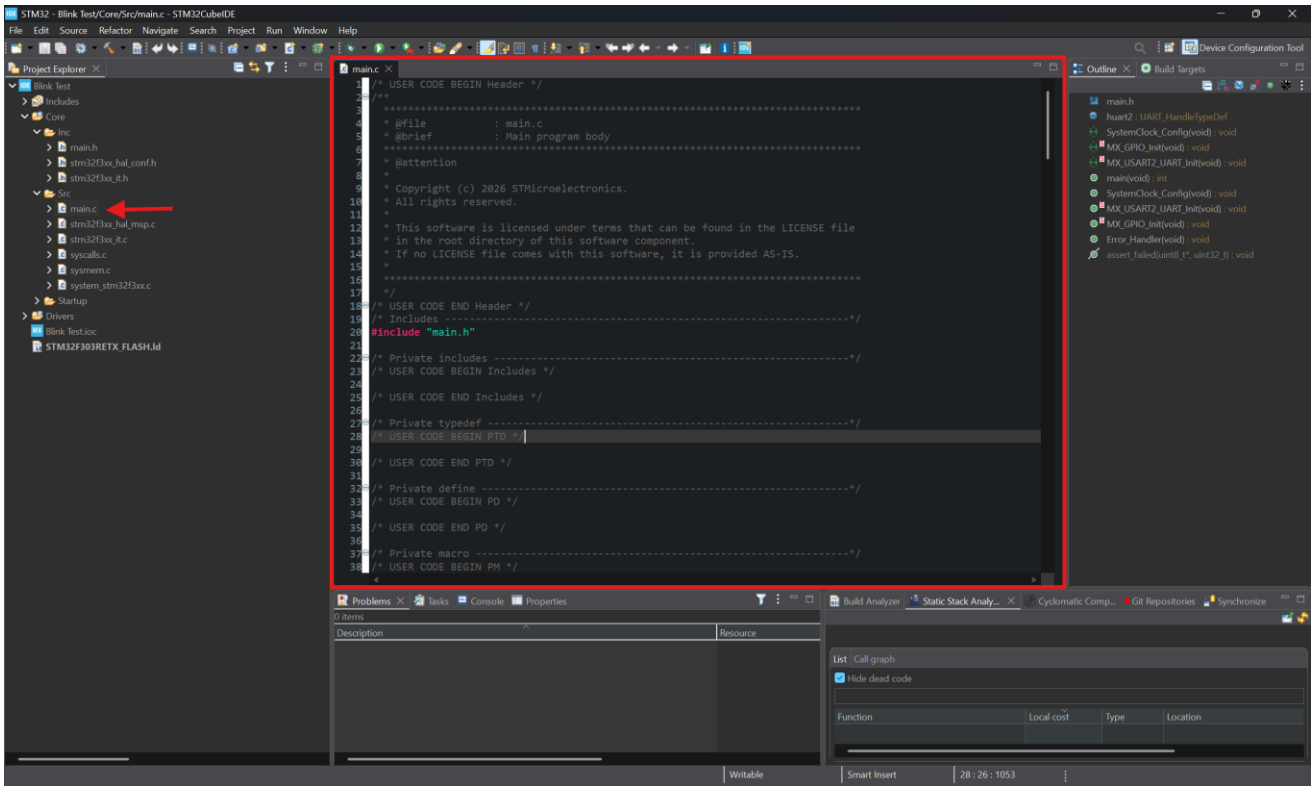




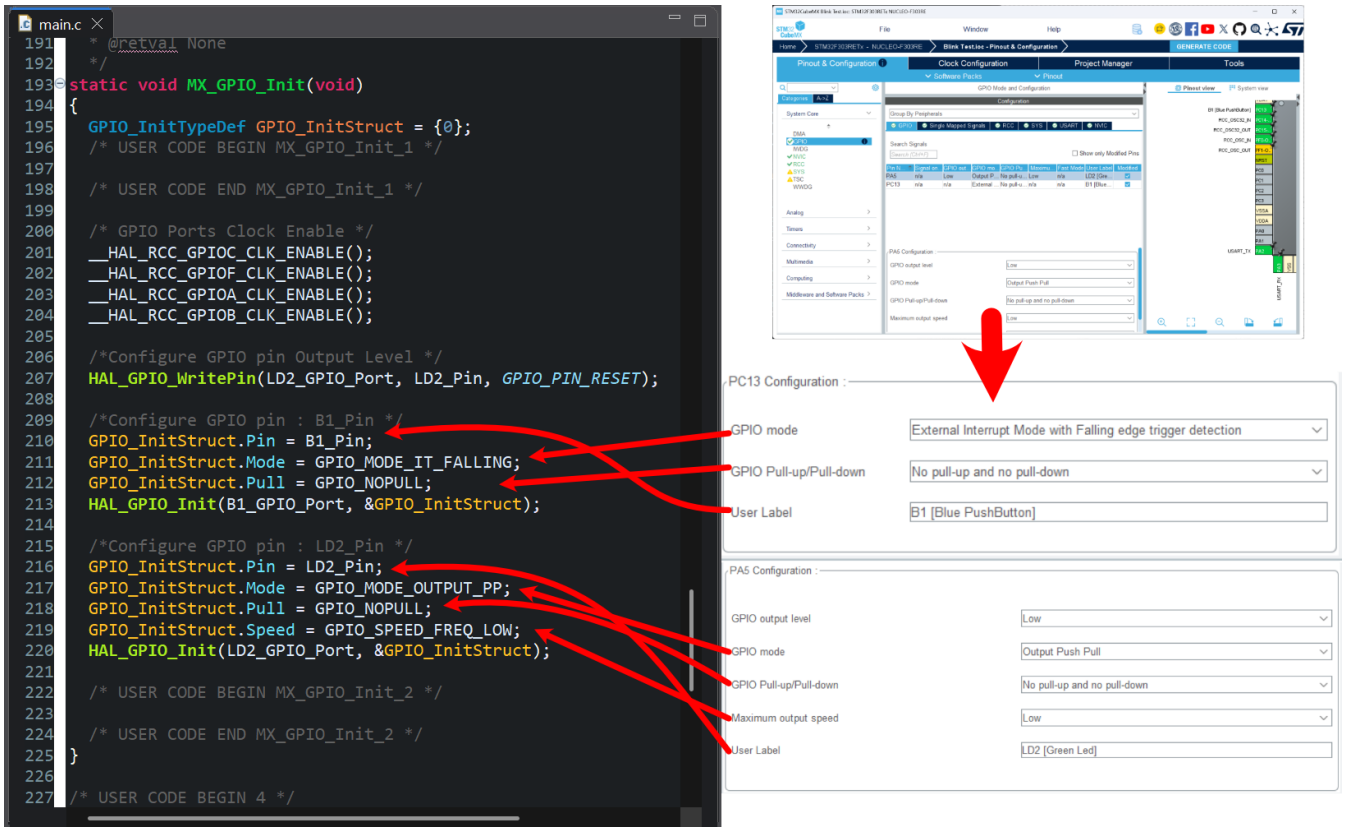
Click '**Directory...**' and select the project parent folder. The project should pop up auto selected, press '**Finish**'.



In the left-hand explorer, expand the '**src**' subdirectory of the project and double click '**main.c**'.



If you scroll through the main file, you will see some existing code. This is code that CubeMX generates. All those configurations with GPIO and pin assignments that we saw in CubeMX are contained within this file. See the below configuration from the CubeMX GUI to the actual code.



Notice how CubeMX is just a graphical interface for generating initialization code. Anytime you change something in CubeMX, you must regenerate the code. Because this is where your main program will be, CubeMX needs to know what code to keep, and what it can regenerate. For this, CubeMX places comments like the ones below in any places where you may want to add your own code. Note that you ***CANNOT*** delete these comments and ***MUST*** put all your code within them (between the begin and end statements) or else it may get deleted when regenerating the initialization code from CubeMX.



This is the main code function. Like normal C, the program starts with the main function. This function is run once and contains the main while loop.

This is your initialization and setup code. This will run once

This is your main while loop code. This will run continuously forever

3.4 Writing the Project Code

Let's begin by looking at these two functions that we will need to blink the LED.

HAL_GPIO_TogglePin

Function name	**void HAL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)**
Function description	Toggle the specified GPIO pin.
Parameters	• **GPIOx:** where x can be (A..F) to select the GPIO peripheral for STM32F3 family • **GPIO_Pin:** specifies the pin to be toggled.
Return values	• **None:**

(UM1786, p220)

HAL_Delay

Function name	**void HAL_Delay (uint32_t Delay)**
Function description	This function provides accurate delay (in milliseconds) based on variable incremented.
Parameters	• **Delay:** specifies the delay time length, in milliseconds.
Return values	• **None:**
Notes	• In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals where uwTick is incremented. The function is declared as __Weak to be overwritten in case of other implementations in user file.

(UM1786, p48-49)

Note: If you go to the doc and take a look above these functions in the same section, you can find information about initialization and using the peripheral, but remember, CubeMX handles the initialization for us so we don't need to!

Looking at these function definitions, we can conclude that for a 0.5s delay, we will need the statement 'HAL_Delay(500);'.

Next, we can look in the original CubeMX project and see that the LED is connected to pin PA5 (UM1724, p24). So, to toggle the LED, our statement will be 'HAL_GPIO_TogglePin(GPIOA, GPIO_Pin_5)'.

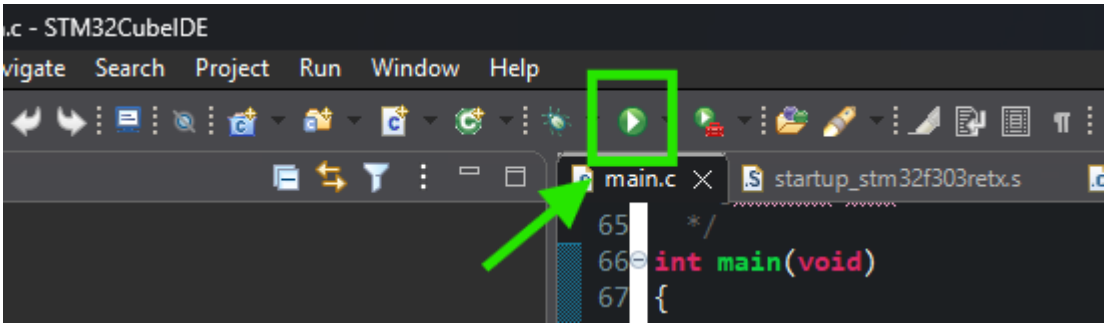
Combining these into the while loop, we get this: (remember, keep your code in-between the begin and end comments)

```
/* USER CODE BEGIN WHILE */
while (1)
{
  HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  HAL_Delay(500);
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

3.5 Running the Code

Plug into the USB connector on the board and click the '**Run**' button on the toolbar.

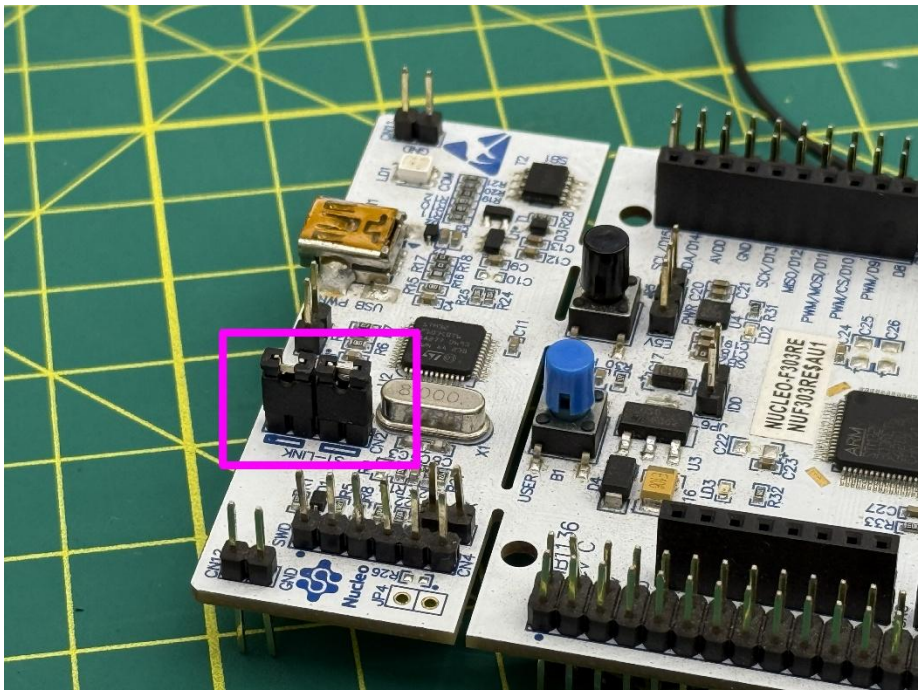


Click '**Ok**' on the edit config dialog. After a few seconds, the project should build and be uploaded to the board and should start running.

3.6 Troubleshooting

Some common issues can be solved with these steps:

1. Check to make sure board is connected to computer
2. Red debug light should illuminate when powered
3. Ensure that the configuration header is correctly bridged



4. Test USB cable to ensure it transfers power and data
5. Try a separate STM32 board
6. Try remaking the project incase any steps were skipped (in particular check for the toolchain config)

4 Expanding the STM32 Project: Button Toggle for Blinking Light

Let's extend this project by having a button to control whether the light is flashing or not. Push it once and it will start the flashing, push it again and it will stop flashing. This will be broken down into multiple steps as this is a bit more complex.

4.1 Simple Button Toggle

We'll start by just having a simple toggle of the light on/off when pushing the button. For this, we will need to read the current state of the button (and thus the pin connected to the button). For this, we will use the following function:

HAL_GPIO_ReadPin

Function name	**GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)**
Function description	Read the specified input port pin.
Parameters	• **GPIOx:** where x can be (A..F) to select the GPIO peripheral for STM32F3 family • **GPIO_Pin:** specifies the port bit to read. This parameter can be GPIO_PIN_x where x can be (0..15).
Return values	• **The:** input port pin value.

(UM1786, p219)

This function will tell us whether the button is being currently pressed. Note that it will not store whether the button **has** been pressed, only if it is **currently** (this will be important later). Looking at the project configuration again, we see that the button is connected to PC13 meaning that our statement will be 'HAL_GPIO_ReadPin (GPIOC, GPIO_PIN_13)'. We can do a simple test if this pin is active using an if statement as seen below: (notice that the ReadPin is negated with the ! because the button on the Nucleo is wired active low)

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if (!HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

If we run this code, we can see that the light responds to our button push, but in a weird and unintended way. This is because our 'HAL_GPIO_TogglePin' function gets repeatedly called for the entire time the button is held down, meaning that each button push actually calls that function multiple times leading to this weird behavior. What we want is for it to only activate once. To do this, we can use a flag to determine whether we have already responded to this button push or not.

For this we are going to initialize a flag variable of type 'uint8_t' meaning that this variable will only take a single byte (the minimum for a variable, of which we only really need two states). Then what we are going to do is when the button is pushed AND the light has not been toggled (i.e. the flag variable has not been set) we will toggle it and set

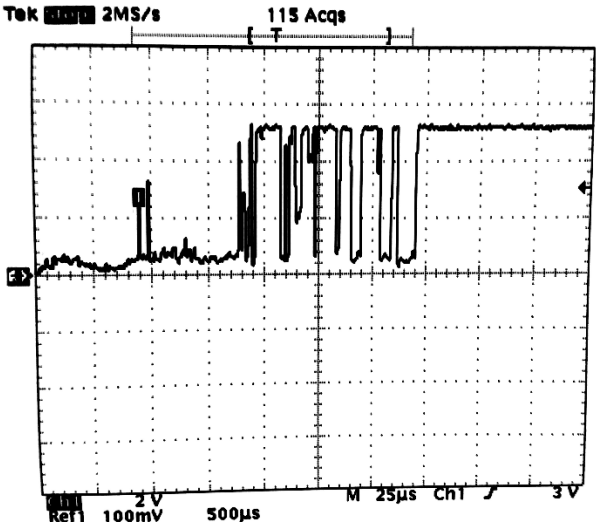
the flag. When the code comes around again, it will see the button still pushed, but the flag set, and thus the function will not be called again.

But what about when we release and push the button again? We want that to activate it again. This brings us to the else if statement. Basically, if the button is no longer pushed and the flag is set, we will clear the flag and reset the ability for the button to activate the toggle function again. See the complete code below:

```
/* USER CODE BEGIN WHILE */
uint8_t light_toggled_flag = 0;
while (1)
{
  if (!HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) && !light_toggled_flag) {
      light_toggled_flag = 1;
      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  } else if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) && light_toggled_flag) {
      light_toggled_flag = 0;
  }
  /* USER CODE END WHILE */
```

4.2 Better Simple Button Toggle: Debouncing

One major thing when working with buttons is to consider something called bouncing. When you push a button, what happens as you push the button is the contacts “bounce” and rapidly connect and disconnect leading to extra button actuations when you did not intend. If you look at the signal below, this is an example of what button bounce looks like:



(Image from here)

Notice the rapid up and down between the button being depressed and pressed. This can cause issues if the number of times a button is pushed matters. In our case, it can cause rapid toggling of the light when you only intended a single actuation. This brings in the concept of *debouncing*, basically, how can we remove these phantom actuations to produce a clean signal. There are both hardware and software methods to debounce the signal, but for our case, we must rely on software to handle the debounce.

One of the ways that we can perform this is through waiting when a signal is triggered. Basically, if we receive an actuation signal, we will ignore any additional signals for a few ms to let that noise pass by. For this we can use ‘HAL_Delay()’ which we used before.

Usually, you would want to measure the amount of time using an oscilloscope to determine the shortest amount of time that still gets past all the noise. For our purposes, we'll assume 1ms is enough (feel free to increase if weird behavior is still exhibited). We'll add a delay where we read the pin, and to make the code a bit more readable, we'll create a variable to hold the state of the button too.

```
/* USER CODE BEGIN WHILE */
uint8_t light_toggled_flag = 0;
uint8_t button_state = 0;
while (1)
{
  button_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
  HAL_Delay(1);

  if (!button_state && !light_toggled_flag) {
      light_toggled_flag = 1;
      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  } else if (button_state && light_toggled_flag) {
      light_toggled_flag = 0;
  }
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

4.3 Re-adding the Blink

Alright, now that we have toggling working, let's add the blink back. Now, because the current 'HAL_GPIO_TogglePin' is only ran once for each button push, we cannot just add the delay there. Instead, we can use another flag. This will indicate whether the blinking should be running or not. To do this, instead of toggling when we push the button, we are going to toggle the flag. Because our flag is 0 for off and 1 for on, we can do a simple bitwise XOR to flip the right-most bit. This is what 'blink_active_flag ^= 1;' is doing. The final 'if' statement handles the actual blinking as long as the flag is set.

```
/* USER CODE BEGIN WHILE */
uint8_t light_toggled_flag = 0;
uint8_t blink_active_flag = 0;
uint8_t button_state = 0;
while (1)
{
  button_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
  HAL_Delay(1);

  if (!button_state && !light_toggled_flag) {
      light_toggled_flag = 1;
      blink_active_flag ^= 1;
  } else if (button_state && light_toggled_flag) {
      light_toggled_flag = 0;
  }

  if (blink_active_flag) {
      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
      HAL_Delay(500);
  }
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

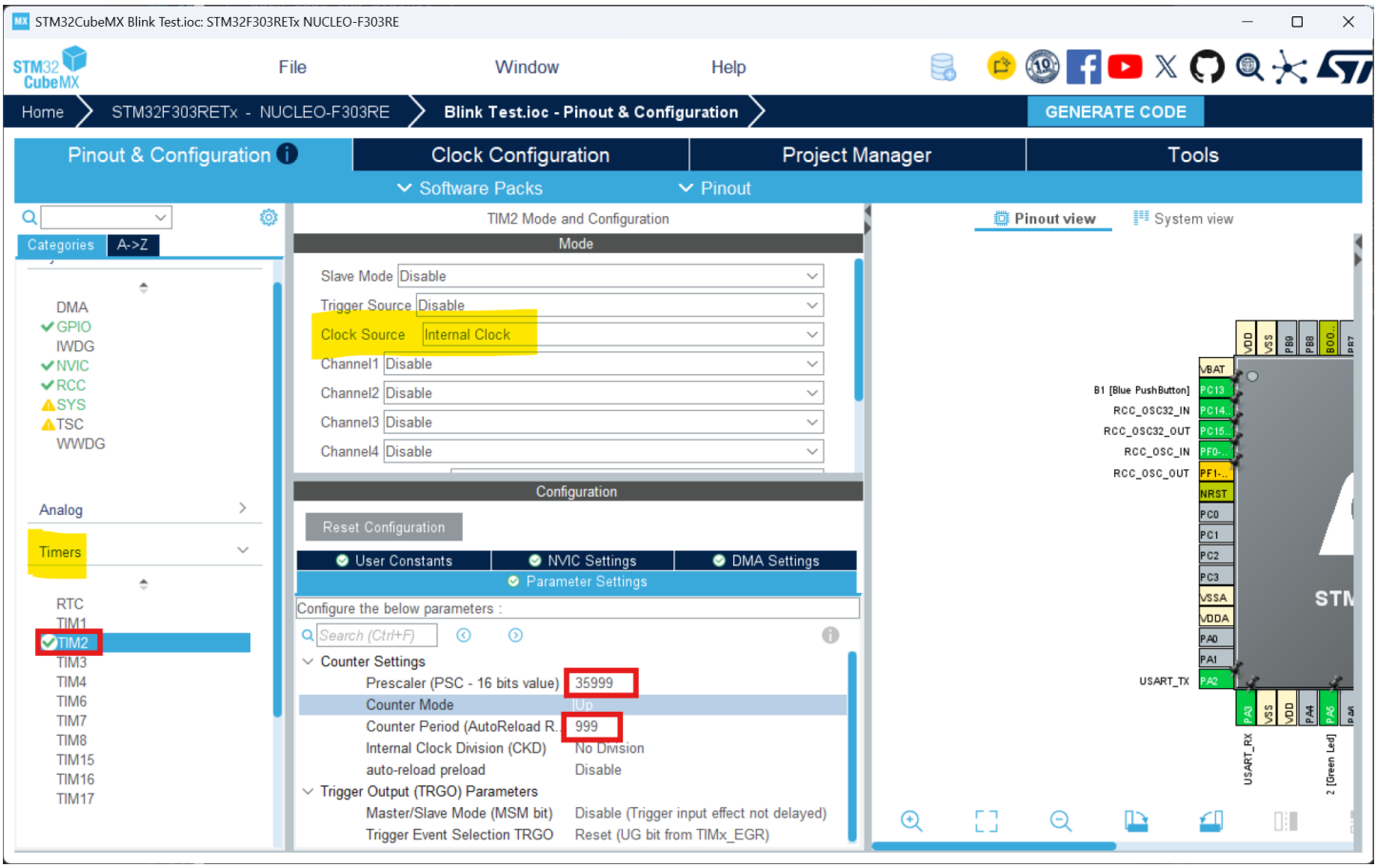
Something to notice though: The button doesn't always stop the blinking, depending on when in the program and how long you hold the button, determines whether the flag gets toggled off or not.

4.4 Fixing the Blink: Timers

Long story short, the 'HAL_Delay' of 500ms is our issue, what is happening is when we call 'HAL_Delay', the CPU is busy waiting, while it is waiting, the CPU cannot process anything else, meaning that if we push the button during that 500ms period, that actuation will not be caught. Instead, we need some way of keeping track of time without having to tie up the CPU so that the MCU can constantly check the button for actuation.

To do this, we will have to use a fundamental feature of MCUs called Timers. For our purposes, timers are basically a variable which automatically increments based on a clock input. This is helpful as it occurs completely separate from the main CPU code. To do this, we will go back to CubeMX.

Start by going to '**Timers**' on the left side, select '**TIM2**'. Under '**Mode and Configuration**' set '**Clock Source**' to '**Internal Clock**'. Set '**Prescaler**' to 35999, and '**Counter Period**' to 999. Remember to click '**Generate Code**'.



Each of these options control important parameters of the timer:

1. **Clock Source**: This is where the trigger to increment the counter is going to come from. Every time the clock is triggered, the timer is incremented by 1.
2. **Prescaler**: This is a hardware divider that sits between the clock source, and the actual counter and is used to slow down how fast the counter increments. For the STM32F303, the clock runs at 75 MHz and depending on the size of the counter (either 16-bit or 32-bit), you could run into an overflow very quickly, so

you use a prescaler to "divide" the incoming clock signal. Say you have a prescaler of 10, for every 10 clock pulses, the counter will increase by 1. For our example, 35,999 means that the clock must pulse 36,000 times before the counter increases by 1 (Note: think of the +1 being how many *extra* times the clock must pulse, so for example, a value of 0 means that the counter will increase every clock pulse)

3. **Counter Period**: This is the max value of the counter. Once the timer hits this value, it will loop back to the bottom again. For us, this means that the clock will tick up until 999 (from 0) and then jump back to the beginning. This prevents us from having to use a modulo operator in the code below. Also, this will can be used to trigger an interrupt, but this will be for later. See the code after the following section.

4.4.1 Timer Math

Now, a value like 35999 sounds like a random value, but it is important, let's look at how we got this number. Firstly, the prescaler register is unsigned 16-bit totaling a max value of 65,535. Next, the most important equation given in the documentation is the following: **(RM0316, p672)**

$$f_{CK_CNT} = \frac{f_{CK_INP}}{PSC + 1}$$

Where:

f_{CK_CNT}: The frequency at which the counter will increase (Hz)

f_{CK_INP}: The input clock frequency (Hz)

PSC: The prescaler value

Moving this function around a bit, we get the following:

$$PSC = \frac{f_{CK_PSC}}{f_{CK_CNT}} - 1$$

If we consider the value that we want to count to, 500ms, it would be appropriate to choose a timer that counts up every 1ms. So, let's fill out the equation.

First let's figure out our desired frequency:

$$f_{CK_CNT} = \frac{1}{T} = \frac{1}{0.001s} = 1\ kHz$$

Next, put our values into the above equation:

$$PSC = \frac{f_{CK_PSC}}{f_{CK_CNT}} - 1 = \frac{72\ MHz}{1\ kHz} - 1 = 71{,}999$$

Note, there is a problem with this number, it is too large to fit in 16-bits. This is why we originally started with 1ms, instead of jumping to like 100ms or even 500ms, our prescaler cannot scale that much. We can work with this, but we will have to decrease our time scale. Let's do 0.5ms instead:

$$f_{CK_CNT} = \frac{1}{T} = \frac{1}{0.0005s} = 2\ kHz$$

$$PSC = \frac{f_{CK_PSC}}{f_{CK_CNT}} - 1 = \frac{72\ MHz}{2\ kHz} - 1 = 35{,}999$$

This number is small enough to fit into our prescaler and means that our counter will increase every 0.5ms. Perfect, but now we want the counter to max out when we hit 500ms. To do this, we do the following:

$$COUNTER\ PERIOD\ (ARR) = (T_{Desired} \times f_{CK_CNT}) - 1$$

$$COUNTER\ PERIOD\ (ARR) = 500ms \times 2\ kHz - 1 = 500 \times 2 - 1 = 999$$

And our counter period is less than the max of that counter (again, either 16 or 32-bits). Perfect, we have our two values! What this means is that the counter will increase every 0.5ms and will top out at 500ms.

4.4.2 Code!

So back to the code, a quick housekeeping thing first. We are going to rename the variable 'light_toggled_flag' to 'button_handled_flag' as that part of the code no longer directly controls the light blink. Next, we are going to have to investigate two more functions:

HAL_TIM_Base_Start

Function name	**HAL_StatusTypeDef HAL_TIM_Base_Start (TIM_HandleTypeDef * htim)**
Function description	Starts the TIM Base generation.
Parameters	• **htim:** TIM Base handle
Return values	• **HAL:** status

(UM1786, p509)

__HAL_TIM_GET_COUNTER	**Description:** • Get the TIM Counter Register value on runtime. **Parameters:** • __HANDLE__: TIM handle. **Return value:** • 16-bit: or 32-bit value of the timer counter register (TIMx_CNT)

(UM1786, p550)

The first function starts the timer, the second function (actually a macro) retrieves the current counter value. Both take the handle of the timer 'htim2' so our two functions will be 'HAL_TIM_Base_Start(&htim2);' and '__HAL_TIM_GET_COUNTER(&htim2);'.

With these, we will put the timer start in the initialization part just before the while loop to get the timer running. Then we will do some basic logic to determine if the timer is at its max value. Like how we use a flag with the button, we don't want multiple activations if we happen to re-read the counter again before the next 0.5ms. To do this, we add the 'timer_handled_flag' which will require the timer to loop back before it resets and allows for the next activation.

We will put the original 'HAL_GPIO_TogglePin' inside the timer handler and remove the delay function as the timer will handle all the timing for us.

Note: One thing to improve this is instead of hardcoding the top of the timer, we can use the macro '__HAL_TIM_GET_AUTORELOAD' to grab the max dynamically, see if you can figure this out yourself. The actual functioning of the code will be the exact same.

```
/* USER CODE BEGIN WHILE */
uint8_t button_handled_flag = 0;
uint8_t blink_active_flag = 0;
uint8_t timer_handled_flag = 0;
uint8_t button_state = 0;

HAL_TIM_Base_Start(&htim2);

while (1)
{
  button_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
  HAL_Delay(1);

  if (!button_state && !button_handled_flag) {
      button_handled_flag = 1;
      blink_active_flag ^= 1;
  } else if (button_state && button_handled_flag) {
      button_handled_flag = 0;
  }

  if (__HAL_TIM_GET_COUNTER(&htim2) == 999 && !timer_handled_flag) {
      timer_handled_flag = 1;
      if (blink_active_flag) {
          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
      }
  } else if (__HAL_TIM_GET_COUNTER(&htim2) != 999 && timer_handled_flag) {
      timer_handled_flag = 0;
  }


  /* USER CODE END WHILE */

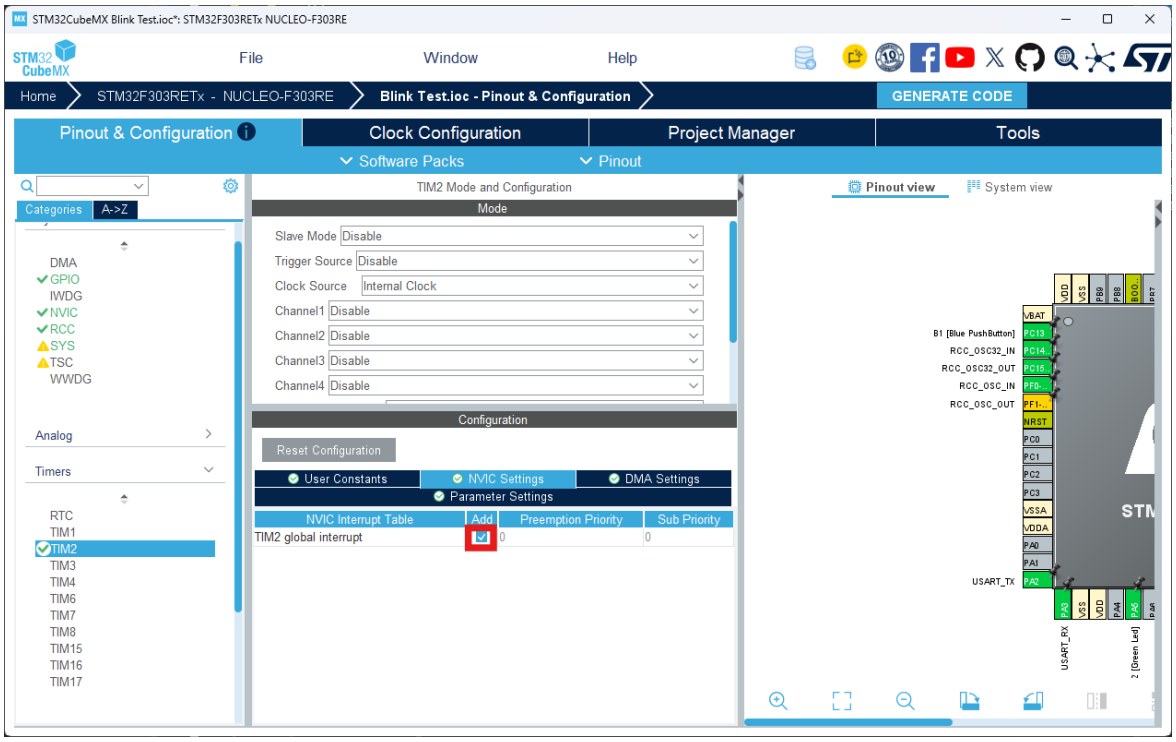
  /* USER CODE BEGIN 3 */
}
```

Notice how when we run this code, we no longer get any issues with the 'HAL_Delay' function blocking further processing, and the button is now responsive no matter where in the code we are. One thing though, this is kind of inefficient constantly polling the counter to see if the counter has reached the top. Theres a better way, in the next section with interrupts!

4.5 Improving the Blink: Interrupts

Instead of the CPU constantly checking if the counter is at the max value, let's instead have the timer tell the CPU when the max value has been reached. This way the CPU is freed up to do other work, and the CPU is only being used when absolutely necessary.

To start, we are going to go back to our CubeMX, and under '**TIM2**' → '**NVIC Settings**' and check '**Add**' next to the '**TIM2 global interrupt**'. Click '**Generate Code**'.



Now, with interrupts, when the interrupt is raised, a callback function is called. We need to define the callback function in our code to indicate what we want to happen when the interrupt is triggered. To do this, we need to override the default callback function 'HAL_TIM_PeriodElapsedCallback'. We are going to place this function definition in our main file. In addition, we are also going to move our global variable definitions above the main function so we can use them in the callback. Lastly, we are removing the timer flag as we no longer need this. The interrupt will only trigger the function once and won't accidentally retrigger the logic.

Going over this callback function, we see that it will receive a TIM handle variable. We can use this in the if statement to determine which timer triggered the interrupt (see the 'htim->Instance == TIM2'). Lastly, we will copy our original 'blink_active_flag' if statement too. Notice the 'volatile' keyword, this is imperative. Basically, what it does is prevent the compiler from optimizing this variable. What can end up happening is the interrupt can be called at any time, and the compiler could have accidentally optimized it to where it won't work right (Google it for more info). Rule of thumb, anytime you have a variable that is used in both this interrupt callback and non-interrupt code, make it volatile. If it is only in this interrupt, or non-interrupt, you do not need volatile (unless it is MMIO, but that's for another time).

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
uint8_t button_handled_flag = 0;
volatile uint8_t blink_active_flag = 0;
uint8_t button_state = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        if (blink_active_flag) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }
    }
}
/* USER CODE END 0 */
```

Jumping down to the while loop, we can first remove the timer logic as that is now handled by the interrupt. Additionally, now that we are using interrupts, we must use a slightly different start function: 'HAL_TIM_Base_Start_IT(&htim2);'.

HAL_TIM_Base_Start_IT

Function name	**HAL_StatusTypeDef HAL_TIM_Base_Start_IT (TIM_HandleTypeDef * htim)**
Function description	Starts the TIM Base generation in interrupt mode.
Parameters	• **htim:** TIM Base handle
Return values	• **HAL:** status

(UM1786, p509)

```
/* USER CODE BEGIN WHILE */

HAL_TIM_Base_Start_IT(&htim2);

while (1)
{
  button_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
  HAL_Delay(1);

  if (!button_state && !button_handled_flag) {
      button_handled_flag = 1;
      blink_active_flag ^= 1;
  } else if (button_state && button_handled_flag) {
      button_handled_flag = 0;
  }

  /* USER CODE END WHILE */

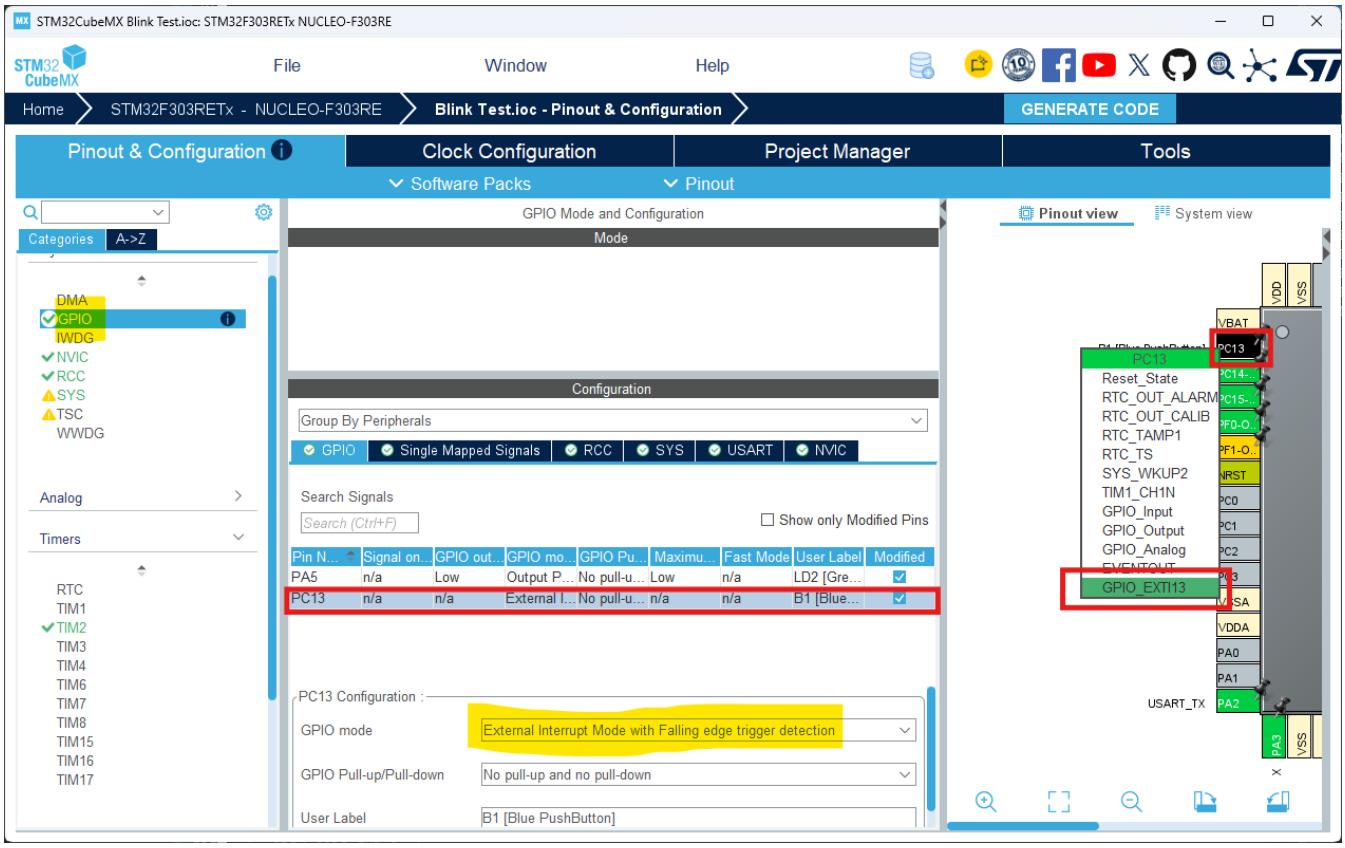
  /* USER CODE BEGIN 3 */
}
```

The code should work the same as before, but this time, we aren't having to constantly poll the timer, rather the timer will interrupt the CPU when it is ready to toggle the light.

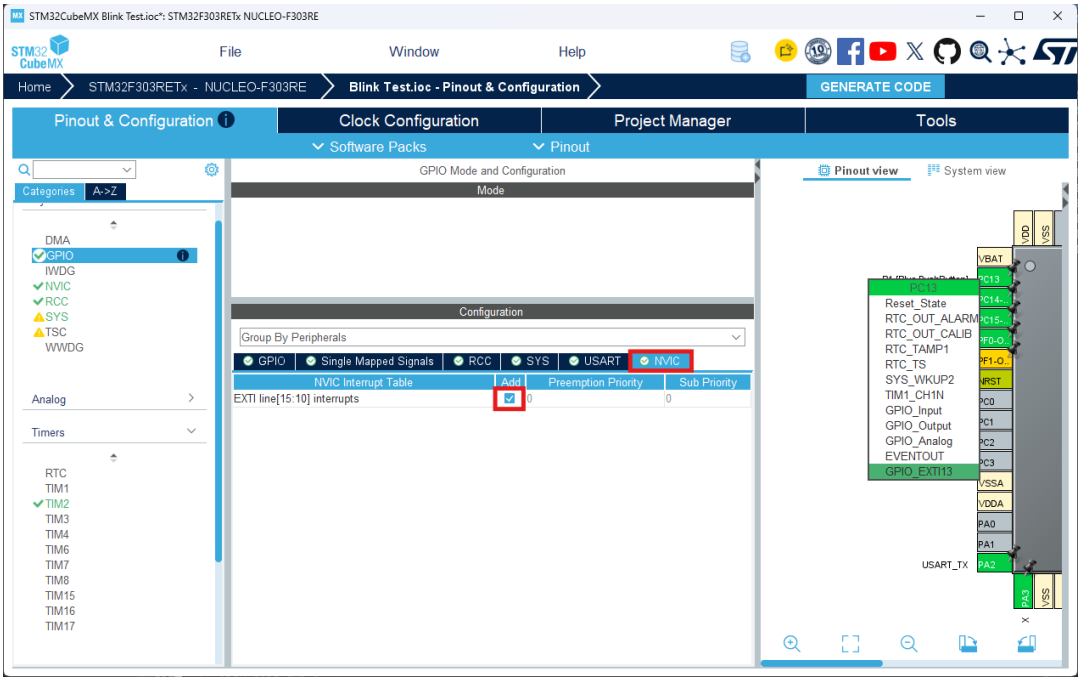
4.6 Interrupts Everywhere! Button GPIO Interrupt

Now that we know the basic operation of interrupts, why don't we do the same thing for the button? This is worth it because we are constantly checking the state of the button in the while loop, this can waste precious CPU time and additionally can miss button actuations if we have much more complex code running requiring more time. An interrupt will make sure that doesn't happen.

Let's go back to CubeMX. Left click on the button pin and change it from '**GPIO_Input**' to '**GPIO_EXTI13**' if it isn't already. Then go to GPIO and click on the button pin under '**Configuration**'. Set the GPIO mode to '**External Interrupt Mode with Falling edge trigger detection**' if not already set. What this means is that an external interrupt (GPIO interrupt) will be triggered when a falling edge is detected (basically when the signal goes from high to low, for instance, when the button is being pressed).



Next, go to the '**NVIC**' page and check the box under '**Add**' and next to '**EXTI line[15:10] interrupts**'. Regenerate the code.



Now for this, we only need to write the callback (no starting function needed like the timer). For this we will override the 'HAL_GPIO_EXTI_Callback' function. Now because we selected falling edge only detection, our current method

for debouncing will not work as it relies on reading when the button is depressed. We'll move to a more advanced method of doing this but first the code without the debounce.

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
volatile uint8_t blink_active_flag = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        if (blink_active_flag) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_13) {
        blink_active_flag ^= 1;

    }
}
/* USER CODE END 0 */
```

```
/* USER CODE BEGIN WHILE */

HAL_TIM_Base_Start_IT(&htim2);

while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Try this code. It should work (minus the bouncing) as normal. Notice how the while loop is completely empty now, we have freed it to do whatever we want. Just remember that the interrupts are still run on the CPU, but only when they need to be (like responding to the button or the timer) rather than all the time.

4.7 Adding the Debounce Back

Let's look at a better way of doing debounce. A better way is to treat an incoming signal as an actuation and ignore any following signals for a set duration (10ms in our case). Then, because we can have bounces on the depressed side of the signal, we store a simple state variable that indicates the next true actuation should be ignored, even if it is more than our set duration away.

For this to work reliably, we must change the interrupt trigger condition to be on both rising and falling edges. If we do not do this, an ideal button press with no outgoing bounces will cause the state to misalign. *I am sure there are better ways to handle debouncing, but this is just what I cooked up at 7pm that doesn't use timers. If you do have an available timer, you can make a much more reliable debounce, but that's for another time.*

Go back to CubeMX and change the '**GPIO mode**' to '**External Interrupt Mode with Rising/Falling edge trigger detection**'

Then we are going to explore the function 'HAL_GetTick();'. This function returns a time, in ms, since the MCU was last reset (or powered up). We can use this to keep track of time without having to use 'HAL_Delay();'.

HAL_GetTick

Function name	**uint32_t HAL_GetTick (void)**
Function description	Povides a tick value in millisecond.
Return values	• **tick:** value
Notes	• The function is declared as __Weak to be overwritten in case of other implementations in user file.

(UM1786, p49)

Basically, the following code will first check to make sure it is the button calling the interrupt. Then we grab the current tick value is and get the difference since the original actual actuation time. Next, we compare to see if it's less than 10ms. If so, we activate the state machine which determines whether the button is being pushed, or depressed and what we should do about it.

Note: It is not advisable to blindly update the state machine like this, but for now it is fine. Additionally, you could move the line which updates the last actuation time, to after the if statement so that even bounces update the time. This means that it will not allow another signal from the last bounce within 10ms rather than the first actuation.

Also, the while loop doesn't change so the screenshot has been omitted.

```c
/* USER CODE BEGIN 0 */
volatile uint8_t blink_active_flag = 0;
volatile uint32_t last_actuation_time = 0;
volatile uint8_t prev_button_state = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        if (blink_active_flag) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        }
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_13) {
        uint32_t current_time = HAL_GetTick();

        if ((current_time - last_actuation_time) > 10) {
            last_actuation_time = current_time;
            if (prev_button_state == 0) {
                blink_active_flag ^= 1;
                prev_button_state = 1;
            } else {
                prev_button_state = 0;
            }
        }
    }
}


/* USER CODE END 0 */
```

One last thing before we wrap up this project. It is decently inefficient to have the timer running when we don't need to be blinking (as this is the only thing the timer is for). We are calling the interrupt and running the comparison for the flag. What we can do instead is remove this 'blink_active_flag' and instead directly turn on and off the timer. This will maximize our free CPU time to do whatever else we want to. Try this out and see if you can get it working.

5 Conclusion

We have gone over the basics of what an STM32 is, what documentation is relevant, and a basic program involving buttons, LEDs, timers, interrupts, and debouncing. This should serve as a jumping off point to get started with more complex projects. It is imperative that you feel comfortable looking in the documentation as there are too many features and functions to write about them all. Hopefully with this guide, you feel you have the tools and ability to run off with STM32 and create what you want.

Some future guides will also be written to go over more advanced topics such as I^2C, SPI, CAN, Clocks, etc... But they will not be as handholding as this one. The handholding takes a lot of writing and time, and is generally something that should be avoided after this guide. There were a few crucial things that were not covered such as version control, commenting, etc.. that will be covered in the next guide. Generally best practices around these are similar to normal coding, so use your best judgement.

Lastly, a complete project is zipped next to this document in case you need to test the board itself. I include this statement here rather than at the beginning to not act as a crutch to lean on, but rather as a tool to compare your code against. If you see this before you finish the project, good job reading until the end, your discipline is much better than mine!